

Desafio Oliveira Trust

Back-End

1. Estruturação do Projeto

- ☐ Criar projeto Spring Boot
 - ☐ Utilizar o Spring Initializr
 - ☐ Incluir dependências: Spring Web, Spring Data (JPA ou driver NoSQL), (opcional) Spring Security, (opcional) Apache POI para Excel
- ☐ Configuração do Banco de Dado
 - ☐ Escolher entre relacional ou NoSQL (MongoDB)
 - ☐ Ajustar `application.properties` (ou `application.yml`)
- ☐ **Configuração de Ferramentas Auxiliares** (opcional)
 - ☐ Cache (Redis)
 - ☐ Mensageria (RabbitMQ)
 - ☐ Container (Docker, docker-compose)

2. Modelagem das Entidades (Domain)

- ☐ Entidade de Registro do Arquivo
 - ☐ Exemplo de campos: `id`, `fileName`, `uploadDate`, `referenceDate`, etc.
- ☐ Entidade do Conteúdo do Arquivo
 - ☐ Exemplo de campos: `id`, `rptDt`, `tckrSymb`, `mktNm`, `sctyCtgyNm`, `isin`, `crpnNm`

3. Repositórios (Persistence Layer)

- ☐ Usar `MongoRepository`

4. Serviços (Service Layer)

- ☐ Serviço de Upload de Arquivo
 - ☐ Validar se o arquivo já existe e se o formato é CSV/Excel
 - ☐ Ler e converter cada linha em `FileContent`
 - ☐ Salvar instâncias no banco em lote

- ☐ Registrar metadados (nome do arquivo, data do upload)
- ☐ Serviço de Histórico de Upload
 - ☐ Buscar uploads por nome do arquivo ou data de referência
 - ☐ Retornar lista e detalhes (ex.: quantidade de registros)
- ☐ Serviço de Consulta ao Conteúdo
 - ☐ Filtros por `tckrSymb` e `rptDt` (opcionais)
 - ☐ Retorno paginado se não houver parâmetros
 - ☐ Retornar JSON com campos solicitados

5. Controladores (API Layer)

- ☐ Endpoint de Upload
 - ☐ POST `/api/v1/files/upload`
 - ☐ Recebe `MultipartFile`, chama serviço de upload
- ☐ Endpoint de Histórico
 - ☐ GET `/api/v1/files/history`
 - ☐ Parâmetros opcionais: `fileName`, `referenceDate`
- ☐ Endpoint de Busca de Conteúdo
 - ☐ GET `/api/v1/files/content`
 - ☐ Parâmetros opcionais: `tckrSymb`, `rptDt`
 - ☐ Se vazio, retorna todos de forma paginada

6. Tratamento de Erros e Exceções

- ☐ **Exception Handling** com `@ControllerAdvice`
 - ☐ Ex.: arquivo duplicado, formato inválido, etc.
 - ☐ Retornar status HTTP apropriado (400, 409, 500)
- ☐ **Logs** de erro, debug, info

7. Possíveis Melhorias (Bônus)

- ☐ **Autenticação e Autorização** (Spring Security, JWT, OAuth2)
- ☐ **Cache** (Spring Cache, Redis)
- ☐ **Filas** (RabbitMQ, Kafka)

- ☐ Processamento assíncrono do upload
- ☐ Containers
 - ☐ Dockerfile, docker-compose
- ☐ Testes
 - ☐ Unitários (Service, lógica de parsing)
 - ☐ Integração (com banco, endpoints)
 - ☐ Carga (75.000+ linhas)
 - ☐

8. Documentação

- ☐ OpenAPI/Swagger
 - ☐ Springdoc ou Springfox para gerar documentação
- ☐ README
 - ☐ Instruções de execução, acesso e teste
- ☐ Exemplos de Requisições
 - ☐ cURL, JSON de entrada/saída, códigos de status